Memoria a corto, medio y largo plazo con RAG

Tutorial básico y detallado para dotar de memoria a un asistente basado en un modelo de lenguaje

Nivel: básico · Lenguaje: Python · Herramientas: uv, ChromaDB, sentence-transformers

Octubre de 2026

4.4 Configurar la clave de API (opcional) 10

Si el índice aparece vacío, haga clic derecho sobre él y elija «Actualizar campos».

1. Introducción

1.1 El problema: los modelos de lenguaje no recuerdan

Un modelo de lenguaje (LLM) no guarda nada entre una llamada y la siguiente. Cada vez que se le envía una petición, el modelo solo «ve» el texto que se incluye en esa petición. Si un asistente parece recordar lo que se dijo hace cinco minutos, es porque el programa que lo usa vuelve a enviarle la conversación completa en cada turno.

Ese enfoque funciona mientras la conversación es corta, pero tiene tres límites claros:

- **Ventana de contexto limitada.** El modelo solo admite una cantidad máxima de texto por petición. Una conversación larga termina por no caber.
- **Coste y latencia.** Cuanto más texto se envía, más se paga y más tarda la respuesta.
- **Nada sobrevive entre sesiones.** Al cerrar el programa, todo lo hablado se pierde, salvo que se guarde en algún sitio.

La solución es construir un **sistema de memoria** fuera del modelo: un programa que decide qué guardar, dónde guardarlo y qué recuperar en cada turno para incluirlo en la petición.

1.2 Qué aprenderá en este tutorial

Se construirá un asistente de consola en Python con tres niveles de memoria:

- **Corto plazo:** los últimos mensajes de la conversación, enviados tal cual.
- **Medio plazo:** resúmenes de la sesión actual y de las sesiones de los últimos días, recuperados con RAG y con un filtro de fecha.
- **Largo plazo:** hechos duraderos sobre el usuario y sus proyectos, extraídos automáticamente y recuperados con RAG.2. Conceptos

2.1 Tokens y ventana de contexto

Los modelos no leen palabras sino **tokens**, fragmentos de texto de unos 3 o 4 caracteres de media. La **ventana de contexto** es el número máximo de tokens que el modelo puede procesar en una sola petición, sumando la entrada y la respuesta. Todo lo que el modelo «sabe» en un momento dado tiene que caber en esa ventana.

2.2 Embeddings

Un **embedding** es una lista de números (un vector) que representa el significado de un texto. Lo genera un modelo especializado, distinto del LLM que conversa. La propiedad clave es esta: **textos con significado parecido producen vectores cercanos**, aunque no compartan palabras.

Por ejemplo, «Ana programa en Python» y «¿Qué lenguaje usa Ana?» tendrán vectores próximos, mientras que «Hoy llueve en Cali» quedará lejos de ambos.

2.3 Base de datos vectorial

Una **base de datos vectorial** guarda textos junto con sus embeddings y permite hacer búsquedas por similitud: dado un texto de consulta, devuelve los k textos guardados cuyo significado es más parecido. En este tutorial se usa **ChromaDB**, que funciona como una biblioteca de Python y guarda los datos en una carpeta local, sin necesidad de instalar un servidor.

Además del texto y el vector, cada registro puede llevar **metadatos** (fecha, importancia, tipo…), que sirven para filtrar los resultados.

2.4 Distancia coseno

Para medir lo «cerca» que están dos vectores se usa la **distancia coseno**. En ChromaDB toma valores entre 0 y 2: un valor cercano a 0 significa que los textos dicen prácticamente lo mismo, y valores altos indican que no tienen relación. En el código se convierte en una **similitud** con la fórmula similitud = 1 - distancia.

2.5 RAG: generación aumentada por recuperación

RAG (Retrieval-Augmented Generation) es la técnica de buscar información relevante en una base de datos y añadirla al prompt antes de llamar al LLM. Tiene dos fases, que se muestran en la figura 1:

1. **Indexación:** cuando hay algo que recordar, se calcula su embedding y se guarda en la base vectorial.
2. **Recuperación:** cuando llega una pregunta, se calcula su embedding, se buscan los fragmentos más parecidos y se insertan en el prompt.

Figura 1. Las dos fases de RAG: indexación y recuperación.

En este tutorial, RAG no se usa para consultar documentos externos sino para **consultar los propios recuerdos del asistente**. Esa es la idea central de la memoria a medio y largo plazo.

3. Los tres niveles de memoria

El diseño se inspira en la memoria humana: lo que se acaba de oír se recuerda literalmente; lo de hace unos días, en forma de resumen; y lo verdaderamente importante se conserva durante años como hechos sueltos. La figura 2 muestra cómo se relacionan los tres niveles.

Figura 2. Visión general: tres memorias que alimentan un único prompt.

Característica	Corto plazo	Medio plazo	Largo plazo
Qué guarda	Mensajes literales recientes	Resúmenes de sesiones	Hechos concretos y duraderos
Duración	Minutos (la sesión actual)	Días o semanas	Meses o años
Dónde vive	Una lista en memoria RAM	Base vectorial, colección «sesiones»	Base vectorial, colección «hechos»
Cómo se recupera	Se envía completa	RAG con filtro de fecha	RAG con puntuación combinada
Cómo se llena	Cada mensaje entra solo	Resumiendo lo que sale del corto plazo	Extrayendo hechos con el LLM
Cómo se olvida	Ventana deslizante	Caducidad por fecha	Borrado explícito o por baja relevancia
Ejemplo	«¿Y en la versión 2?»	«El lunes se revisó el informe de ventas»	«Ana vive en Cali y prefiere Python»

3.1 Por qué no basta con un solo nivel

- **Solo corto plazo:** el asistente olvida todo al cerrar la sesión, y las conversaciones largas no caben.
- **Solo largo plazo:** se pierde el hilo inmediato. Preguntas como «¿y la segunda opción?» necesitan los mensajes literales de hace un momento.
- **Sin medio plazo:** no hay forma de responder a «¿en qué quedamos ayer?», porque el detalle de ayer no es un hecho permanente pero tampoco está ya en la conversación.

Idea clave La información fluye de un nivel al siguiente: los mensajes que salen del corto plazo se resumen en el medio plazo, y de la conversación se extraen hechos para el largo plazo. Este proceso se llama consolidación.

4. Preparación del entorno

4.1 Instalar uv

uv es un gestor de proyectos de Python muy rápido. Instala Python si hace falta, crea el entorno virtual y gestiona las dependencias con un solo comando. Abra una terminal y ejecute el instalador correspondiente a su sistema.

En Windows (PowerShell):

powershell -ExecutionPolicy ByPass -c “irm https://astral.sh/uv/install.ps1 | iex”

En macOS o Linux:

curl -LsSf https://astral.sh/uv/install.sh | sh

Cierre la terminal, abra una nueva y compruebe la instalación con uv --version.

4.2 Crear el proyecto e instalar las dependencias

uv init memoria-rag
cd memoria-rag
uv add chromadb anthropic sentence-transformers

Cada dependencia cumple una función concreta:

- chromadb: la base de datos vectorial.
- sentence-transformers: genera los embeddings con un modelo multilingüe que funciona bien en español.
- anthropic: cliente para llamar a un LLM. Puede sustituirse por el de otro proveedor cambiando un solo archivo.

Tenga en cuenta sentence-transformers instala PyTorch, que ocupa varios cientos de megas. La primera instalación puede tardar unos minutos. La primera vez que se ejecute el programa se descargará el modelo de embeddings (unos 470 MB). Las siguientes veces se usa la copia local.

4.3 Estructura final del proyecto

Al terminar el tutorial, la carpeta tendrá estos archivos. uv init crea un main.py de ejemplo que se sustituirá en el paso 7.

```
memoria-rag/
├── pyproject.toml ← creado por uv
├── llm.py ← paso 1: conexión con el modelo
├── almacen.py ← paso 2: base vectorial
├── memoria_corto.py ← paso 3
├── memoria_medio.py ← paso 4
├── memoria_largo.py ← paso 5
├── gestor.py ← paso 6: coordina las tres memorias
├── main.py ← paso 7: chat de consola
└── datos/
└── chroma/ ← se crea solo al ejecutar
```

4.4 Configurar la clave de API (opcional) {#4.4-configurar-la-clave-de-api-(opcional)}

Si dispone de una clave de API, defínala como variable de entorno en la misma terminal donde ejecutará el programa:

```
# Windows (PowerShell)
$env:ANTHROPIC_API_KEY = “su-clave-aquí”

# macOS / Linux
export ANTHROPIC_API_KEY=“su-clave-aquí”
```

Nunca escriba la clave dentro del código ni la suba a un repositorio. Si no define la variable, el programa funciona en **modo simulado**: responde repitiendo parte del mensaje, lo que permite comprobar que la memoria a corto plazo y el almacén funcionan, aunque no se generarán resúmenes ni hechos reales.

5. Paso 1: conexión con el modelo de lenguaje

Todas las memorias necesitan llamar al LLM, ya sea para conversar, para resumir o para extraer hechos. Centralizar esa llamada en un solo archivo permite cambiar de proveedor sin tocar el resto del código. Cree el archivo llm.py:

llm.py

```
“””Conexión con el modelo de lenguaje (LLM).”””
import os
from pathlib import Path
from typing import Any

from dotenv import load_dotenv

BASE_DIR = Path(__file__).resolve().parents[2]
load_dotenv(BASE_DIR / “.env”)

MODELO = “claude-sonnet-5-5”  # Cambie el modelo según su proveedor

def llamar_llm(sistema: str, mensajes: list[dict[str, Any]], max_tokens: int = 1024) -> str:
    “””Envía un prompt de sistema y una lista de mensajes al LLM y devuelve el texto.”””
    if not mensajes:
        raise ValueError(“La lista de mensajes no puede estar vacía.”)

    if os.environ.get(“ANTHROPIC_API_KEY”):
        import anthropic

        cliente = anthropic.Anthropic()
        respuesta = cliente.messages.create(
            model=MODELO,
            max_tokens=max_tokens,
            system=sistema,
```

```
        messages=mensajes,
    )

    textos = []
    for bloque in getattr(respuesta, "content", []):
        if hasattr(bloque, "text"):
            textos.append(bloque.text)
    if textos:
        return "".join(textos)
    return ""

# Modo de prueba: sin clave de API devuelve una respuesta simulada
ultimo_mensaje = mensajes[-1]
contenido = ultimo_mensaje.get("content", "")

if isinstance(contenido, list):
    piezas = []
    for item in contenido:
        if isinstance(item, dict):
            pieza = item.get("text", str(item))
            piezas.append(str(pieza))
        else:
            piezas.append(str(item))
    contenido = " ".join(piezas)

return "[respuesta simulada]" + str(contenido)[:200]
```

La función recibe dos cosas: un **prompt de sistema** (las instrucciones generales y la memoria recuperada) y una **lista de mensajes** con el formato {"role": "user" | "assistant", "content": "..."}. Devuelve únicamente el texto de la respuesta.

Cambiar de proveedor Si usa otro proveedor, solo debe reescribir el bloque que está dentro del if, manteniendo la misma firma de la función. Consulte la documentación de su proveedor para conocer el nombre exacto del modelo y actualice la constante MODELO.

6. Paso 2: el almacén vectorial

Las memorias de medio y largo plazo comparten la misma base vectorial, pero cada una usa su propia **colección** (el equivalente a una tabla). Cree el archivo almacen.py:

almacen.py

```
"""Almacén vectorial compartido por las memorias de medio y largo plazo."""
import chromadb
from chromadb.utils import embedding_functions

def crear_cliente(ruta: str = "datos/chroma") -> chromadb.ClientAPI:
"""Crea (o abre) una base vectorial persistente en disco."""
return chromadb.PersistentClient(path=ruta)
```

```
def funcion_embeddings():
"""Modelo multilingüe: funciona bien con textos en español."""
return embedding_functions.SentenceTransformerEmbeddingFunction(
model_name="paraphrase-multilingual-MiniLM-L12-v2"
)

def obtener_coleccion(cliente, nombre: str, funcion_emb=None):
"""Devuelve una colección que mide la similitud con la distancia coseno."""
return cliente.get_or_create_collection(
name=nombre,
embedding_function=funcion_emb or funcion_embeddings(),
metadata={"hnsw:space": "cosine"},
)
```

6.1 Qué hace cada función

- crear_cliente abre la base de datos en la carpeta datos/chroma. Si no existe, la crea. Al ser **persistente**, los datos sobreviven al cerrar el programa.
- funcion_embeddings indica qué modelo convierte el texto en vectores. Se usa paraphrase-multilingual-MiniLM-L12-v2, que entiende más de 50 idiomas, entre ellos el español.
- obtener_coleccion devuelve una colección configurada para medir la similitud con la **distancia coseno** (hnsw:space: cosine).

Importante El modelo de embeddings debe ser siempre el mismo para una colección. Si lo cambia, los vectores antiguos dejan de ser comparables con los nuevos y tendrá que borrar la carpeta datos/chroma para empezar de cero.

---

7. Paso 3: memoria a corto plazo

7.1 Concepto: la ventana deslizante

La memoria a corto plazo es la más simple: una lista con los últimos mensajes. Cuando la lista supera un límite, los mensajes más antiguos salen por un extremo mientras los nuevos entran por el otro. Es lo que se llama una **ventana deslizante** (figura 3).

Figura 3. Ventana deslizante: los mensajes antiguos salen hacia la memoria a medio plazo.

El detalle importante es que los mensajes que salen **no se tiran**: se devuelven para que la memoria a medio plazo los resuma. Así no se pierde información, solo se comprime.

7.2 Implementación

Cree el archivo memoria_corto.py:

memoria_corto.py

```
"""Memoria a corto plazo: los últimos mensajes de la conversación."""
```

class MemoriaCortoPlazo:
def __init__(self, max_mensajes: int = 10, max_caracteres: int = 8000):
self.max_mensajes = max_mensajes
self.max_caracteres = max_caracteres
self.mensajes: list[dict] = []

```
 def \_tamano(self) \-\> int:
     return sum(len(m\["content"\]) for m in self.mensajes)

 def agregar(self, rol: str, contenido: str) \-\> list\[dict\]:
     """Añade un mensaje y devuelve los mensajes que salen de la
ventana."""
     self.mensajes.append({"role": rol, "content": contenido})
     expulsados \= \[\]
     while (len(self.mensajes) \> self.max\_mensajes
            or self.\_tamano() \> self.max\_caracteres) and
len(self.mensajes) \> 2:
         \# Se expulsan en pares (usuario \+ asistente) para no romper
la alternancia
         expulsados.extend(self.mensajes\[:2\])
         self.mensajes \= self.mensajes\[2:\]
     return expulsados

 def obtener(self) \-\> list\[dict\]:
     return list(self.mensajes)

 def vaciar(self) \-\> list\[dict\]:
     restantes, self.mensajes \= self.mensajes, \[\]
     return restantes
```

7.3 Explicación

- Se aplican **dos límites**: número de mensajes (max_mensajes) y tamaño total en caracteres (max_caracteres). El segundo evita que unos pocos mensajes muy largos llenen la ventana de contexto.
- Los mensajes se expulsan **de dos en dos** (pregunta y respuesta). Muchos LLM exigen que la conversación empiece por un mensaje del usuario y alterne con el asistente; expulsar por pares mantiene esa regla.
- La condición len(self.mensajes) > 2 garantiza que nunca se vacíe por completo: siempre queda al menos el último intercambio.
- vaciar() se usa al cerrar la sesión para enviar los mensajes restantes al resumen.

8. Paso 4: memoria a medio plazo

8.1 Concepto: resumen progresivo y sesiones recientes

La memoria a medio plazo tiene dos partes que trabajan juntas:

1. **Resumen de la sesión actual.** Cada vez que la ventana de corto plazo expulsa mensajes, se pide al LLM que los integre en un resumen que va creciendo. A esto se le llama **resumen progresivo** (figura 4). El resumen se incluye en el prompt para que el asistente no pierda el hilo de lo que se habló al principio de la sesión.

2. **Resúmenes de sesiones anteriores.** Al cerrar la sesión, el resumen se guarda en la base vectorial con la fecha. En sesiones futuras se recupera con RAG, pero **solo si es reciente** (por defecto, de los últimos 14 días).

Figura 4. Resumen progresivo: el resumen anterior y los mensajes expulsados producen un resumen actualizado.

8.2 Implementación

Cree el archivo memoria_medio.py:

memoria_medio.py

```
“””Memoria a medio plazo: resumen de la sesión actual y resúmenes de sesiones recientes.”””
import time
import uuid

from llm import llamar_llm

PROMPT_RESUMEN = (
“Usted mantiene un resumen breve de una conversación. Integre los mensajes nuevos”
“en el resumen existente. Conserve decisiones, tareas pendientes, datos concretos”
“y preferencias. Elimine saludos y repeticiones. Máximo 150 palabras.”
)

def formatear(mensajes: list[dict]) -> str:
return “\n”.join(f”{m[‘role’]}: {m[‘content’]}” for m in mensajes)

class MemoriaMedioPlazo:
def __init__(self, coleccion, dias_vigencia: int = 14):
self.coleccion = coleccion
self.dias_vigencia = dias_vigencia
self.resumen_sesion = “”
self.id_sesion = str(uuid.uuid4())
```

```
 def incorporar(self, mensajes: list\[dict\]) \-\> None:
     """Resumen progresivo: integra en el resumen los mensajes que
salen del corto plazo."""
     if not mensajes:
         return
```

```
    contenido \= (f"Resumen actual:\\n{self.resumen\_sesion or
'(vacío)'}\\n\\n"
                  f"Mensajes nuevos:\\n{formatear(mensajes)}")
    self.resumen\_sesion \= llamar\_llm(PROMPT\_RESUMEN, \[{"role":
"user", "content": contenido}\])

 def cerrar\_sesion(self) \-\> None:
    """Guarda el resumen de la sesión en la base vectorial, con
fecha."""
    if not self.resumen\_sesion:
        return
    self.coleccion.add(
        ids=\[self.id\_sesion\],
        documents=\[self.resumen\_sesion\],
        metadatas=\[{"timestamp": time.time(), "tipo": "resumen\
_sesion"}\],
    )
    self.resumen\_sesion \= ""
    self.id\_sesion \= str(uuid.uuid4())

 def recuperar(self, consulta: str, k: int \= 3, distancia\_max: float
\= 0.6) \-\> list\[str\]:
    """RAG con filtro temporal: solo resúmenes de los últimos N
días."""
    if self.coleccion.count() \== 0:
        return \[\]
    limite \= time.time() \- self.dias\_vigencia \* 86400
    r \= self.coleccion.query(
        query\_texts=\[consulta\],
        n\_results=min(k, self.coleccion.count()),
        where={"timestamp": {"\$gte": limite}},
    )
    return \[doc for doc, dist in zip(r\["documents"\]\[0\], r\
["distances"\]\[0\])
            if dist \<= distancia\_max\]

 def purgar\_caducados(self) \-\> None:
    """Borra los resúmenes que superan el periodo de vigencia."""
    limite \= time.time() \- self.dias\_vigencia \* 86400
    self.coleccion.delete(where={"timestamp": {"\$lt": limite}})
```

8.3 Explicación

- **`PROMPT_RESUMEN`** indica al LLM qué conservar (decisiones, tareas, datos) y qué descartar (saludos, repeticiones). El límite de 150 palabras impide que el resumen crezca sin control.

- **`incorporar`** combina el resumen actual con los mensajes nuevos y sustituye el resumen por la versión actualizada.

- **`cerrar_sesion`** guarda el resumen en la colección con el metadato timestamp (segundos desde 1970) y prepara un identificador nuevo para la siguiente sesión.
- **`recuperar`** es RAG con un **filtro de metadatos**: where={"timestamp": {"$gte": limite}} hace que ChromaDB solo busque entre los resúmenes posteriores a la fecha límite. Además, descarta los resultados con distancia superior a distancia_max, para no incluir sesiones que no tienen relación con la pregunta.
- **`purgar_caducados`** borra definitivamente los resúmenes antiguos. Puede llamarse, por ejemplo, una vez al día.

¿Por qué filtrar por fecha? Sin el filtro, un resumen de hace un año podría aparecer en el prompt solo porque comparte palabras con la pregunta actual. El filtro garantiza que el medio plazo sea, de verdad, «reciente». Lo que deba durar más tiempo tiene que pasar a la memoria a largo plazo en forma de hecho.

---

9. Paso 5: memoria a largo plazo

9.1 Concepto: hechos atómicos

La memoria a largo plazo no guarda conversaciones sino **hechos**: frases cortas, independientes y en tercera persona, como «El usuario se llama Ana» o «El proyecto usa PostgreSQL 16». Guardar hechos pequeños en lugar de párrafos tiene dos ventajas: la búsqueda por similitud es más precisa y es fácil actualizar o borrar un dato concreto.

La figura 5 muestra el ciclo de vida completo de un hecho.

Figura 5. Ciclo de vida de un hecho: extracción, deduplicación, almacenamiento, recuperación, refuerzo y olvido.

9.2 Implementación

Cree el archivo memoria_largo.py:

memoria_largo.py

```
"""Memoria a largo plazo: hechos duraderos recuperados con RAG."""
import json
import math
import time
import uuid

from llm import llamar_llm

PROMPT_EXTRACCION = (
"Extraiga de la conversación los hechos que conviene recordar de forma permanente:"
"datos del usuario, preferencias estables, decisiones y conocimientos importantes."
"Ignore lo trivial o temporal. Responda SOLO con un JSON válido con esta forma:"
'[{"hecho": "texto breve en tercera persona", "importancia": 1-10}].'
```

“Si no hay nada que recordar, responda [].”
)

class MemoriaLargoPlazo:
def __init__(self, coleccion, umbral_duplicado: float = 0.15):
self.coleccion = coleccion
self.umbral_duplicado = umbral_duplicado

```
 def guardar\_hecho(self, hecho: str, importancia: int \= 5\) \-\>
str:
     """Guarda un hecho. Si ya existe uno casi idéntico, lo
actualiza."""
     ahora \= time.time()
     id\_hecho \= str(uuid.uuid4())
     if self.coleccion.count() \> 0:
         r \= self.coleccion.query(query\_texts=\[hecho\], n\
_results=1)
         if r\["distances"\]\[0\] and r\["distances"\]\[0\]\[0\] \<
self.umbral\_duplicado:
             id\_hecho \= r\["ids"\]\[0\]\[0\]  \# se reutiliza el id:
actualización
     self.coleccion.upsert(
         ids=\[id\_hecho\],
         documents=\[hecho\],
         metadatas=\[{"importancia": importancia, "creado": ahora,
"ultimo\_acceso": ahora}\],
     )
     return id\_hecho

 def extraer\_y\_guardar(self, mensajes: list\[dict\]) \-\> list\
[str\]:
     """Pide al LLM que extraiga hechos de la conversación y los
guarda."""
     texto \= "\\n".join(f"{m\['role'\]}: {m\['content'\]}" for m in
mensajes)
     salida \= llamar\_llm(PROMPT\_EXTRACCION, \[{"role": "user",
"content": texto}\])
     try:
         limpio \= salida.replace("\`\`\`json", "").replace("\`\`\`",
"").strip()
         hechos \= json.loads(limpio)
     except json.JSONDecodeError:
         return \[\]
     guardados \= \[\]
     for h in hechos:
         if isinstance(h, dict) and h.get("hecho"):
             self.guardar\_hecho(h\["hecho"\],
int(h.get("importancia", 5)))
             guardados.append(h\["hecho"\])
     return guardados
```

```
 def recuperar(self, consulta: str, k: int \= 5, candidatos: int \=
15\) \-\> list\[str\]:
     """RAG con puntuación combinada: similitud \+ importancia \+
recencia."""
     total \= self.coleccion.count()
     if total \== 0:
         return \[\]
     r \= self.coleccion.query(query\_texts=\[consulta\], n\
_results=min(candidatos, total))
     ahora \= time.time()
     puntuados \= \[\]
     for id\_, doc, meta, dist in zip(r\["ids"\]\[0\], r\
["documents"\]\[0\],
                                     r\["metadatas"\]\[0\], r\
["distances"\]\[0\]):
         similitud \= 1 \- dist                              \# 1 \=
idéntico
         importancia \= meta.get("importancia", 5\) / 10   \# 0..1
         dias \= (ahora \- meta.get("ultimo\_acceso", ahora)) / 86400
         recencia \= math.exp(-dias / 30\)                 \# decae
con los días
         puntuacion \= 0.6 \* similitud \+ 0.25 \* importancia \+ 0.15
\* recencia
         puntuados.append((puntuacion, id\_, doc, meta))
     puntuados.sort(reverse=True)
     elegidos \= \[p for p in puntuados\[:k\] if p\[0\] \> 0.35\]
     \# Refuerzo: lo que se recuerda se "refresca"
     if elegidos:
         self.coleccion.update(
             ids=\[p\[1\] for p in elegidos\],
             metadatas=\[{\*\*p\[3\], "ultimo\_acceso": ahora} for p
in elegidos\],
         )
     return \[p\[2\] for p in elegidos\]

 def olvidar(self, consulta: str) \-\> int:
     """Borra los hechos muy parecidos a la consulta (derecho al
olvido)."""
     if self.coleccion.count() \== 0:
         return 0
     r \= self.coleccion.query(query\_texts=\[consulta\], n\
_results=3)
     ids \= \[i for i, d in zip(r\["ids"\]\[0\], r\["distances"\]\
[0\]) if d \< 0.3\]
     if ids:
         self.coleccion.delete(ids=ids)
     return len(ids)
```

9.3 Explicación por etapas

Extracción

extraer_y_guardar envía un fragmento de la conversación al LLM con PROMPT_EXTRACCION, que le pide devolver **solo JSON** con los hechos y una importancia del 1 al 10. El código limpia posibles marcas de bloque de código (```json) y, si el JSON no es válido, simplemente no guarda nada: es preferible perder un hecho a que el programa falle.

Deduplicación

Antes de guardar, guardar_hecho busca el hecho más parecido. Si la distancia es menor que umbral_duplicado (0,15), se considera el mismo hecho y se **reutiliza su identificador**. Como se usa upsert (actualizar o insertar), el registro antiguo se sobrescribe en lugar de duplicarse. Así, si el usuario dice dos veces que vive en Cali, solo habrá un registro.

Recuperación con puntuación combinada

Recuperar solo por similitud tiene un problema: un dato trivial pero muy parecido a la pregunta podría desplazar a uno importante. Por eso recuperar trae 15 candidatos y los reordena con una puntuación que combina tres factores:

Factor	Cálculo	Peso	Qué aporta
Similitud	1 – distancia coseno	0,60	Relación con la pregunta actual
Importancia	importancia / 10	0,25	Prioriza datos relevantes (nombre, decisiones)
Recencia	e^(–días / 30)	0,15	Favorece lo que se ha usado hace poco

La recencia usa una curva exponencial: un hecho consultado hoy vale 1; uno consultado hace 30 días, unos 0,37; y uno de hace 90 días, unos 0,05. Solo se devuelven los hechos con puntuación final superior a 0,35.

Refuerzo

Cada vez que un hecho se recupera, se actualiza su metadato ultimo_acceso. Igual que en la memoria humana, **lo que se recuerda con frecuencia se mantiene fresco**, y lo que nunca se usa va perdiendo prioridad.

Olvido

olvidar borra los hechos muy parecidos a un texto. Es útil cuando el usuario pide expresamente que se olvide algo, o cuando un dato deja de ser cierto. Es una función importante desde el punto de vista de la privacidad.

10. Paso 6: el gestor de memoria

El gestor es la pieza que coordina los tres niveles. En cada turno de conversación sigue los ocho pasos de la figura 6.

Figura 6. Flujo completo de un turno de conversación.

10.1 Implementación

Cree el archivo gestor.py:

gestor.py
“““Gestor que coordina los tres niveles de memoria.”””
from almacen import crear_cliente, obtener_coleccion
from llm import llamar_llm
from memoria_corto import MemoriaCortoPlazo
from memoria_largo import MemoriaLargoPlazo
from memoria_medio import MemoriaMedioPlazo

SISTEMA_BASE = “Usted es un asistente útil. Responda en español y de forma concisa.”

class GestorMemoria:
def __init__(self, ruta: str = “datos/chroma”, funcion_emb=None, extraer_cada: int = 6):
cliente = crear_cliente(ruta)
self.corto = MemoriaCortoPlazo(max_mensajes=10)
self.medio = MemoriaMedioPlazo(obtener_coleccion(cliente, “sesiones”, funcion_emb))
self.largo = MemoriaLargoPlazo(obtener_coleccion(cliente, “hechos”, funcion_emb))
self.extraer_cada = extraer_cada
self.pendientes_extraccion: list[dict] = []

```
 def construir\_sistema(self, consulta: str) \-\> str:
     partes \= \[SISTEMA\_BASE\]
     hechos \= self.largo.recuperar(consulta)
     if hechos:
         partes.append("\#\# Lo que sabe del usuario (memoria a largo
plazo)\\n"
                       \+ "\\n".join(f"- {h}" for h in hechos))
     sesiones \= self.medio.recuperar(consulta)
     if sesiones:
         partes.append("\#\# Conversaciones recientes relacionadas
(medio plazo)\\n"
                       \+ "\\n".join(f"- {s}" for s in sesiones))
     if self.medio.resumen\_sesion:
         partes.append("\#\# Resumen de lo hablado antes en esta
sesión\\n"
                       \+ self.medio.resumen\_sesion)
     partes.append("Use esta información solo si es relevante para la
pregunta.")
     return "\\n\\n".join(partes)
```

```
 def conversar(self, mensaje\_usuario: str) \-\> str:
     sistema \= self.construir\_sistema(mensaje\_usuario)
     mensajes \= self.corto.obtener() \+ \[{"role": "user", "content":
mensaje\_usuario}\]
     respuesta \= llamar\_llm(sistema, mensajes)

     for rol, texto in (("user", mensaje\_usuario), ("assistant",
respuesta)):
         expulsados \= self.corto.agregar(rol, texto)
         self.medio.incorporar(expulsados)          \# corto \-\>
medio
         self.pendientes\_extraccion.append({"role": rol, "content":
texto})

     if len(self.pendientes\_extraccion) \>= self.extraer\_cada:
         self.largo.extraer\_y\_guardar(self.pendientes\
_extraccion)  \# \-\> largo
         self.pendientes\_extraccion \= \[\]
     return respuesta

 def cerrar\_sesion(self) \-\> None:
     """Consolida todo al terminar la sesión."""
     restantes \= self.corto.vaciar()
     self.medio.incorporar(restantes)
     if self.pendientes\_extraccion:
         self.largo.extraer\_y\_guardar(self.pendientes\_extraccion)
         self.pendientes\_extraccion \= \[\]
     self.medio.cerrar\_sesion()
```

10.2 Cómo se construye el prompt

construir_sistema arma el prompt de sistema por bloques, en un orden pensado para que el modelo distinga claramente cada tipo de memoria (figura 7). Los bloques vacíos no se incluyen, así que en la primera conversación el prompt es mínimo.

Figura 7. Composición del contenido enviado al LLM en cada turno.

La última instrucción, «Use esta información solo si es relevante», es importante: evita que el asistente mencione datos del usuario cuando no vienen al caso, algo que resulta extraño e incluso incómodo.

10.3 Cuándo se consolida la memoria

- **Corto → medio:** en cada turno, si la ventana expulsa mensajes.
- **Conversación → largo:** cada extraer_cada mensajes (6 por defecto, es decir, tres intercambios). No conviene hacerlo en cada mensaje porque cada extracción es una llamada adicional al LLM.
- **Cierre de sesión:** cerrar_sesion vacía la ventana, completa el resumen, extrae los hechos pendientes y guarda el resumen en la colección de sesiones.

11. Paso 7: programa principal y prueba

Sustituya el contenido de main.py (el que creó uv init) por el siguiente:

main.py

```
“””Chat de consola con memoria a corto, medio y largo plazo.”””
from gestor import GestorMemoria

def main():
gestor = GestorMemoria()
print(“Chat con memoria. Escriba ‘salir’ para terminar.\n”)
while True:
texto = input(“Usted:”).strip()
if texto.lower() in {“salir”, “exit”}:
gestor.cerrar_sesion()
print(“Sesión guardada. Hasta pronto.”)
break
if texto:
print(f”Asistente: {gestor.conversar(texto)}\n”)

if __name__ == “__main__”:
main()
```

Ejecute el programa desde la carpeta del proyecto:

```
uv run main.py
```

11.1 Prueba recomendada

Para comprobar los tres niveles, haga dos sesiones separadas. Las respuestas que se muestran a continuación son ilustrativas; las reales dependerán del modelo.

Primera sesión:

Usted: Hola, me llamo Ana y trabajo como analista de datos en Cali.
Asistente: ¡Mucho gusto, Ana! ¿En qué puedo ayudarle hoy?

Usted: Estoy preparando un informe de ventas trimestral con pandas.
Asistente: Perfecto. ¿Ya tiene los datos limpios o empezamos por ahí?

Usted: Prefiero que los ejemplos siempre sean en Python.
Asistente: Entendido, usaré Python en todos los ejemplos.

Usted: salir
Sesión guardada. Hasta pronto.

Segunda sesión (vuelva a ejecutar uv run main.py):

Usted: ¿Recuerda en qué estaba trabajando?
Asistente: Sí, Ana. Estaba preparando un informe de ventas trimestral con pandas. ¿Quiere continuar con él?

Usted: ¿Cómo agrupo las ventas por mes?
Asistente: Como prefiere Python, aquí tiene un ejemplo con pandas: …

En la segunda sesión, el nombre y la preferencia por Python vienen de la **memoria a largo plazo**, y el informe de ventas viene del **resumen de la sesión anterior** (medio plazo).

11.2 Inspeccionar lo que se ha guardado

Puede ver el contenido de la memoria con este pequeño script, que puede guardar como ver_memoria.py y ejecutar con uv run ver_memoria.py:

ver_memoria.py

```
from almacen import crear_cliente, obtener_coleccion

cliente = crear_cliente()
for nombre in (“hechos”, “sesiones”):
col = obtener_coleccion(cliente, nombre)
print(f”\n=== {nombre} ({col.count()} registros) ===“)
datos = col.get()
for doc, meta in zip(datos[”documents”], datos[“metadatas”]):
print(“-”, doc, meta)
```

12. Ajuste de parámetros

Los valores por defecto son un punto de partida razonable. La siguiente tabla resume qué controla cada parámetro y cuándo conviene cambiarlo.

Parámetro	Archivo	Valor	Efecto al aumentarlo
max_mensajes	memoria_corto.py	10	Más contexto literal, más coste por turno
max_caracteres	memoria_corto.py	8000	Admite mensajes más largos en la ventana
dias_vigencia	memoria_medio.py	14	Recuerda sesiones más antiguas
distancia_max	memoria_medio.py	0,6	Admite sesiones menos relacionadas
umbral_duplicado	memoria_largo.py	0,15	Fusiona hechos menos parecidos (riesgo de perder matices)
k	memoria_largo.py	5	Más hechos en el prompt
Pesos 0,6 / 0,25 / 0,15	memoria_largo.py	—	Cambian la prioridad entre similitud, importancia y

Parámetro	Archivo	Valor	Efecto al aumentarlo
			recencia
extraer_cada	gestor.py	6	Menos llamadas al LLM, pero los hechos tardan más en guardarse

Cómo ajustar con criterio Cambie un solo parámetro cada vez y repita la misma conversación de prueba. Use el script ver_memoria.py para comprobar qué se guarda y añada un print en construir_sistema para ver exactamente qué recibe el modelo en cada turno.

13. Buenas prácticas y errores comunes

13.1 Buenas prácticas

- **No guarde datos sensibles.** Añada al prompt de extracción una instrucción para ignorar contraseñas, números de tarjetas, datos médicos, etc. La memoria se guarda en texto plano en disco.
- **Ofrezca control al usuario.** Permita consultar y borrar lo que el asistente recuerda (por ejemplo, con comandos como «olvida que…»).
- **Trate la memoria como datos, no como instrucciones.** Un texto guardado podría contener frases como «ignora tus reglas». Por eso el gestor la presenta en secciones con títulos claros y como información de referencia.
- **Separe usuarios.** Si el asistente lo usan varias personas, añada un metadato usuario y fíltrelo en cada consulta con where.
- **Registre lo que ocurre.** Guardar en un log qué recuerdos se recuperan en cada turno facilita mucho la depuración.

13.2 Errores comunes

Síntoma	Causa probable	Solución
No se guardan hechos	El LLM no devuelve JSON válido, o está en modo simulado	Imprima la variable salida en extraer_y_guardar y revise el prompt
Hechos duplicados	Umbral de duplicado demasiado bajo	Suba umbral_duplicado a 0,2
Recuerdos irrelevantes en el prompt	Umbrales de recuperación demasiado permisivos	Baje distancia_max o suba el corte de 0,35
El asistente pierde el hilo	Ventana de corto plazo demasiado pequeña	Aumente max_mensajes
Error de dimensiones al	Se cambió el modelo de	Borre datos/chroma y

Síntoma	Causa probable	Solución
consultar	embeddings	empiece de nuevo
La primera ejecución tarda mucho	Descarga del modelo de embeddings	Es normal; solo ocurre una vez

14. Próximos pasos

Con esta base funcionando, estas son algunas mejoras naturales, ordenadas de menor a mayor complejidad:

1. **Memoria por usuario:** añadir un identificador de usuario a los metadatos y filtrar por él.
2. **Resolución de contradicciones:** si llega «Ana se mudó a Bogotá», pedir al LLM que decida si sustituye a «Ana vive en Cali» en lugar de guardar ambos.
3. **Búsqueda híbrida:** combinar la búsqueda por similitud con búsqueda por palabras clave (BM25) para nombres propios y códigos.
4. **Memoria de documentos:** añadir una cuarta colección con fragmentos de documentos propios (manuales, apuntes), que es el uso clásico de RAG.
5. **Evaluación:** preparar un conjunto de conversaciones de prueba con las respuestas esperadas y medir cuántas veces se recupera el recuerdo correcto.

Glosario

Término	Definición
LLM	Modelo de lenguaje de gran tamaño; genera texto a partir de un prompt.
Token	Fragmento de texto que el modelo procesa como unidad.
Ventana de contexto	Máximo de tokens que el modelo procesa en una petición.
Embedding	Vector numérico que representa el significado de un texto.
Base vectorial	Base de datos que almacena embeddings y busca por similitud.
Colección	Grupo de registros dentro de una base vectorial, similar a una tabla.
Metadatos	Datos adicionales de un registro (fecha, importancia) usados para filtrar.
Distancia coseno	Medida de diferencia entre dos vectores; 0 indica significado equivalente.

Término	Definición
RAG	Técnica que recupera información relevante y la añade al prompt antes de generar.
Top-k	Los k resultados más similares de una búsqueda.
Resumen progresivo	Resumen que se actualiza integrando nueva información al anterior.
Consolidación	Paso de información de un nivel de memoria a otro más duradero.
Upsert	Operación que actualiza un registro si existe o lo inserta si no.